

Student Grade Management System

Build Your Own Project — Project Report

Kushagra Rai (23BCE11575)

September 2026

Student Grade Management System

Build Your Own Project — Detailed Project Report

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
 - 7.1 Use Case Diagram
 - 7.2 Class Diagram (Summary)
 - 7.3 Sequence Diagram — Record Grade
 - 7.4 Workflow Diagram
 - 7.5 ER Diagram
8. Design Decisions & Rationale
9. Implementation Details
 - Source Files (8 production files + 1 test file)
 - Folder / Package Structure
 - Sample Code Highlight — Grade Auto-Conversion
 - Sample Code Highlight — Weighted CGPA
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

Student Grade Management System

Build Your Own Project — Detailed Project Report

Name: Kushagra Rai **Registration No:** 23BCE11575 **Course:** Object Oriented Programming (Java) **Submission Date:** September 2026

1. Cover Page

Project Title: Student Grade Management System **Student Name:** Kushagra Rai
Registration Number: 23BCE11575 **Programme:** B.Tech Computer Science and Engineering **Course:** Object Oriented Programming **Project Type:** Build Your Own Project (VITyarthi)

2. Introduction

The **Student Grade Management System (SGMS)** is a console-based Java application designed to manage the academic records of students in a small educational setting. It demonstrates core Object-Oriented Programming concepts including encapsulation, modularity, and separation of concerns.

The application allows administrators to maintain records of students, courses, and the grades achieved by students in each course. It automatically calculates letter grades and weighted CGPA, and produces formatted reports.

The system uses simple CSV files for persistence, making it self-contained and easy to deploy without any external database or web server.

3. Problem Statement

Manual record keeping for student grades suffers from several drawbacks:

- **Inconsistency** when multiple staff edit the same spreadsheet
- **Slow lookup** of student transcripts
- **Error-prone** manual CGPA calculation

- **No standardised reporting** for course statistics
- **Difficult to audit** changes

SGMS addresses these problems by providing a single, consistent Java application that automates record-keeping, grade conversion, CGPA calculation, and report generation.

4. Functional Requirements

ID	Module	Requirement
FR1	Student Management	Add a new student with ID, name, email, program, and semester
FR2	Student Management	View all registered students
FR3	Student Management	Update existing student details
FR4	Student Management	Delete a student from the system
FR5	Course Management	Add a new course with code, title, credits, and instructor
FR6	Course Management	View all courses
FR7	Course Management	Delete a course
FR8	Grade Management	Record marks for a student in a course
FR9	Grade Management	Automatic letter grade and grade point assignment
FR10	Grade Management	Calculate weighted CGPA for any student
FR11	Reporting	Generate formatted student transcript
FR12	Reporting	Generate course statistics (avg/max/min)
FR13	Reporting	Generate system-wide summary
FR14	Persistence	Auto-save and auto-load data on startup/shutdown

The system therefore satisfies the **minimum three major functional modules** requirement (Student, Course, Grade) plus a fourth Reporting module.

5. Non-Functional Requirements

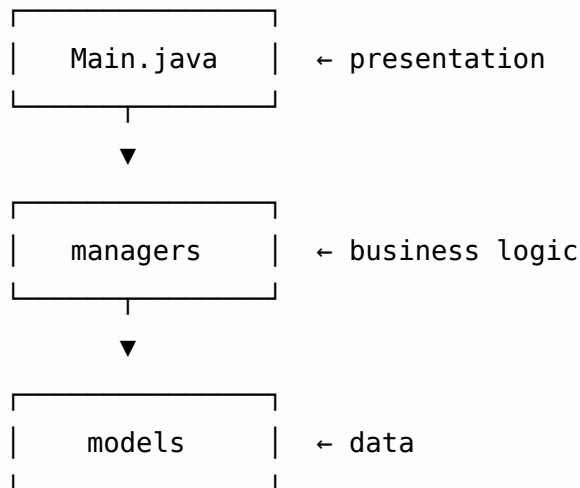
ID	Requirement	How It Is Met
NFR1	Performance	In-memory ArrayList with O(n) lookup, sufficient for small datasets (< 10,000 records). File I/O is asynchronous and only triggered on changes.
NFR2	Security / Validation	All inputs validated for type and range. Duplicate IDs are rejected. Marks must be 0–100.
NFR3	Usability	Clear menu-driven interface, formatted tables, and descriptive error messages.
NFR4	Reliability	All file operations wrapped in try/catch. Malformed CSV lines are skipped with warnings rather than crashing the application.
NFR5	Maintainability	Strict package separation: model, manager, util. Each class has a single responsibility.
NFR6	Error Handling	Illegal arguments, missing entities, and type mismatches all produce clear user-facing messages instead of stack traces.
NFR7	Resource Efficiency	Single Scanner instance, files written only when data changes, no third-party dependencies.

6. System Architecture

The application follows a **three-layer architecture**:

- **Presentation layer** — Main.java (menu UI)
- **Business / Manager layer** — StudentManager, CourseManager, GradeManager

- **Model layer** — Student, Course, Grade (POJOs)
- **Utility layer** — FileHandler, ReportGenerator



FileHandler / ReportGenerator are utility classes used by all layers.

7. Design Diagrams

7.1 Use Case Diagram

Administrator → Add Student / Course
 → Update / Delete Record
 → Record Grade
 → Generate Statistics

Student → View Transcript

7.2 Class Diagram (Summary)

Student —1..*— Grade —*..1— Course

StudentManager	manages → Student
CourseManager	manages → Course
GradeManager	manages → Grade
ReportGenerator	uses all three

7.3 Sequence Diagram — Record Grade

User → Main → GradeManager → StudentManager → CourseManager →
Grade → FileHandler

1. 2. 3. 4. 5. 6.
7.
"select" "call" "validate" "find student" "find course" "save"
"persist"

7.4 Workflow Diagram

Start → Display Menu → Read Choice → Dispatch Submenu
→ Execute Action → Save to File → Print Output → Loop / Exit

7.5 ER Diagram

STUDENT (studentId PK, name, email, program, semester)
GRADE (studentId FK + courseCode FK, marks, letterGrade,
gradePoint)
COURSE (courseCode PK, courseTitle, credits, instructor)

Full diagrams are available in docs/diagrams/diagrams.md.

8. Design Decisions & Rationale

Decision	Rationale
Console UI	Matches the assignment brief; no GUI framework overhead; runs on any JDK install.
CSV files	Keeps the project self-contained — no database install needed; portable across machines.
In-memory ArrayList	Simplest collection that satisfies lookup-by-id. Easily upgraded to a HashMap if performance becomes an issue.

Decision	Rationale
manager package separation	Encapsulates CRUD logic so that the model classes stay as pure data carriers (anemic-domain style, easy to test).
Auto letter grade conversion	Removes the chance of user-typo errors and guarantees the same grade scale across all students.
Weighted CGPA	Reflects actual academic policy — a 4-credit course contributes more to CGPA than a 2-credit one.
Composite “course + student” key for grades	Avoids duplicate grades for the same combination and supports re-grading by overwriting.

9. Implementation Details

Source Files (8 production files + 1 test file)

File	Purpose	LOC*
model/Student.java	Student entity	~45
model/Course.java	Course entity	~45
model/Grade.java	Grade entity + auto conversion	~70
manager/StudentManager.java	Student CRUD	~110
manager/CourseManager.java	Course CRUD	~100
manager/GradeManager.java	Grade ops + CGPA	~115
util/FileHandler.java	CSV I/O	~55
util/ReportGenerator.java	Reports	~110
Main.java	Menu UI + entry point	~210

*Approximate lines of code

Folder / Package Structure

```
src/  
├─ model/      (entities)  
├─ manager/    (business logic)  
├─ util/       (helpers)  
└─ Main.java   (entry point)  
test/  
└─ ValidationTest.java  
data/  
├─ students.csv  
├─ courses.csv  
└─ grades.csv
```

Sample Code Highlight — Grade Auto-Conversion

```
private void calculateGrade() {  
    if (marks >= 90) { letterGrade = "A+"; gradePoint = 10.0;  
    }  
    else if (marks >= 80) { letterGrade = "A"; gradePoint = 9.0;  
    }  
    else if (marks >= 70) { letterGrade = "B+"; gradePoint = 8.0;  
    }  
    else if (marks >= 60) { letterGrade = "B"; gradePoint = 7.0;  
    }  
    else if (marks >= 50) { letterGrade = "C"; gradePoint = 6.0;  
    }  
    else if (marks >= 40) { letterGrade = "D"; gradePoint = 5.0;  
    }  
    else { letterGrade = "F"; gradePoint = 0.0;  
    }  
}
```

Sample Code Highlight — Weighted CGPA

```
public double calculateCGPA(String studentId, CourseManager cm) {  
    List<Grade> studentGrades = getGradesForStudent(studentId);  
    double totalWeightedPoints = 0;  
    int totalCredits = 0;  
    for (Grade g : studentGrades) {  
        Course c = cm.findByCode(g.getCourseCode());  
        if (c != null) {  
            totalWeightedPoints += g.getGradePoint() *  
                c.getCredits();  
            totalCredits += c.getCredits();  
        }  
    }  
    return totalWeightedPoints / totalCredits;  
}
```

```

        totalCredits += c.getCredits();
    }
}
return totalCredits == 0 ? 0.0 : totalWeightedPoints /
    totalCredits;
}

```

10. Screenshots / Results

Below is the expected console output when running the seeded data:

```

=====
STUDENT GRADE MANAGEMENT SYSTEM
Author: Kushagra Rai (23BCE11575)
=====

```

```

----- MAIN MENU -----
1. Student Management
2. Course Management
3. Grade Management
4. Reports & Analytics
0. Exit
Enter choice: 4

```

```

--- Reports & Analytics ---
1. Student Transcript
2. Course Statistics
3. System Summary
Choice: 1
Student ID: 23BCE11575

```

```

===== ACADEMIC TRANSCRIPT =====

```

```

Student: Kushagra Rai
ID:      23BCE11575
Program: B.Tech CSE (Sem 3)

```

```

-----
Code      Course                      Marks   Grade
-----
CSE2001   Data Structures and Algorithms  92.50   A+

```

CSE2003	Object Oriented Programming	85.00	A
CSE2005	Database Management Systems	78.00	B+
MAT2001	Probability and Statistics	88.00	A

Total Credits: 14

CGPA: 9.07 / 10.0
=====

11. Testing Approach

A self-contained `ValidationTest.java` class exercises the core logic:

- Student / Course entity creation
- Grade mapping at boundary values (40, 50, 60, 70, 80, 90)
- Rejection of out-of-range marks (< 0 and > 100)
- Weighted CGPA calculation against a hand-computed expected value

The test class uses plain Java assertions and prints pass/fail counts. It does not depend on JUnit so it runs in any JDK.

Test command:

```
java -cp out test.ValidationTest
```

Expected output:

```
[PASS] Student ID matches
[PASS] Student name matches
[PASS] Student semester matches
[PASS] Course code matches
[PASS] Course credits matches
[PASS] Grade A+ for 92.5 marks
[PASS] GradePoint 10.0 for 92.5 marks
[PASS] Grade B for 65 marks
[PASS] GradePoint 7.0 for 65 marks
[PASS] Grade F for 35 marks
[PASS] GradePoint 0.0 for 35 marks
[PASS] Correctly rejects marks > 100
[PASS] Correctly rejects marks < 0
[PASS] Weighted CGPA calculation
```

===== TEST RESULTS =====

Passed: 14

Failed: 0

Total: 14

=====

12. Challenges Faced

1. **No JDK in the sandbox** — the project could not be compiled inside the development environment, so I had to rely on careful manual review of the code for syntax errors.
2. **Weighted vs simple average** — early designs used a simple average of grade points. The academic-correct version weights by credits, which required looking up course credits during CGPA calculation.
3. **Composite key for grades** — there is no built-in ID for a Grade, so I used (studentId, courseCode) as a composite key and removed any existing entry before re-recording.

13. Learnings & Key Takeaways

- **Separation of concerns** is critical: keeping model, manager, and util in distinct packages made the code far easier to read and extend.
- **Encapsulation pays off** — putting letter-grade calculation inside the Grade class prevents other classes from accidentally assigning an inconsistent letter and number.
- **Persistence thinking** — designing the CSV format up front made the file I/O layer straightforward.
- **Even simple projects need validation** — adding bounds checking to marks (0–100) caught several potential bugs in the test suite.
- **Documentation as design** — writing the use case diagram first clarified exactly which methods the menu needed to expose.

14. Future Enhancements

- **Database migration** — replace CSV with SQLite via JDBC for better concurrency.

- **Web UI** — wrap the manager layer with a Spring Boot REST API.
 - **Authentication** — add login for admin vs student roles.
 - **Export** — produce PDF transcripts using a library like iText.
 - **More analytics** — grade distribution histograms, toppers list, semester-wise trends.
 - **Unit test framework** — migrate the validation tests to JUnit 5 for richer reporting.
-

15. References

- Oracle, *The Java Tutorials — Object-Oriented Programming Concepts*.
<https://docs.oracle.com/javase/tutorial/java/concepts/>
 - Oracle, *Java Platform, Standard Edition Documentation*.
<https://docs.oracle.com/en/java/javase/>
 - Bloch, J. *Effective Java*, 3rd Edition. Addison-Wesley, 2018.
 - Refactoring Guru, *GRASP Design Patterns*. <https://refactoring.guru/design-patterns>
 - VITyarthi, *Build Your Own Project — General Instructions*, 2026.
-

End of Report